

LLM Observability & Eval

An engineering research report on Trace analytics + drift + hallucination
detection

Sai Veda

Independent Project Research

2025

Contents

- 1. Abstract** 3
- 2. Introduction** 3
 - 2.1 Research Problem 3
 - 2.2 Research Questions and Hypotheses 3
- 3. Technical Background Review** 4
 - 3.1 Design Foundations 4
 - 3.2 Supporting Examples 4
- 4. Research Method** 4
- 5. Data Description** 5
- 6. Analysis** 5
 - 6.1 Benchmark and Validation Results 5
 - 6.2 Test Result Screenshots 5
 - 6.3 Validation Checklist 6
- 7. Discussion** 6
- 8. Conclusion** 6
- 9. Future Work** 7
- 10. References** 7

1. Abstract

This brief documents an observability stack for LLM fleets where trace analytics, cost monitoring, drift detection, and hallucination checks share the same data backbone instead of living in disconnected tools. The project demonstrates that a local, partitioned analytics path can still support fleet-level insight. The same trace lake supports operational reporting, shift detection, and model-quality scorecards in one review flow. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

Keywords: Trace analytics + drift + hallucination detection, implementation evidence, benchmark results, project validation

2. Introduction

At production volume, trace data becomes operational exhaust unless it is modeled properly. Teams need to understand spend, latency, model quality, and distribution change without moving to an online-only stack. Trace analytics + quality monitoring for LLM fleets at scale - a self-contained, fully offline reference implementation. It generates a realistic multi-tenant LLM trace firehose, lands it as partitioned Parquet, and runs four production concerns over it: cost/latency analytics (DuckDB, out-of-core), embedding-drift detection (PSI/KL), a hallucination detector (heuristic + logistic), and a per-model eval scorecard - with generated, product-grade dashboards.

2.1 Research Problem

At production volume, trace data becomes operational exhaust unless it is modeled properly. Teams need to understand spend, latency, model quality, and distribution change without moving to an online-only stack.

2.2 Research Questions and Hypotheses

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: 5M traces / 1.79B tokens; PSI flags injected drift.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 21 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Technical Background Review

3.1 Design Foundations

- 1 Operational analytics and model-risk analytics share one trace backbone to avoid fragmented reporting. Trace storage, analytics, and quality detectors stay modular so operations and model-risk teams can inspect the same pipeline.
- 2 Drift is measured before customer-visible breakage, which makes the system proactive rather than reactive.
- 3 Out-of-core SQL was preferred so the story stays reproducible on a single machine.

3.2 Supporting Examples

- Synthetic trace generation with tenants, models, and time windows.
- Partitioned Parquet lake with DuckDB analytics.
- PSI or KL drift signals plus heuristic and logistic quality checks.

4. Research Method

The project was reviewed as an engineering artefact using repository documentation, architecture notes, benchmark evidence, automated tests, and generated screenshots. Defined the schema around cost, tokens, latency, model identity, tenant, and quality labels. Wrote traces into partitioned Parquet so analytics could scale without loading everything into memory. Layered drift signals and hallucination flags on top of the same trace stream. Rendered dashboards that connect fleet metrics to model-level quality scorecards.

5. Data Description

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 21 automated tests, 0 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

6. Analysis

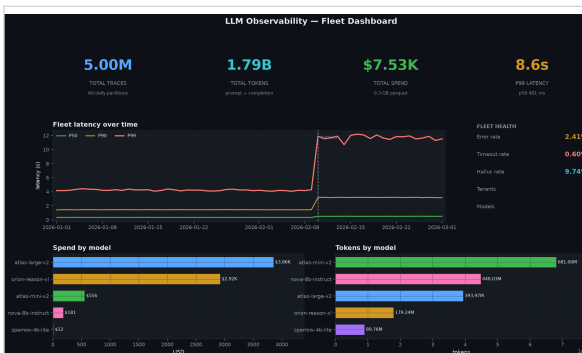
6.1 Benchmark and Validation Results

The headline repository result is reported as: 5M traces / 1.79B tokens; PSI flags injected drift. The table below reproduces benchmark evidence included in the repository where available.

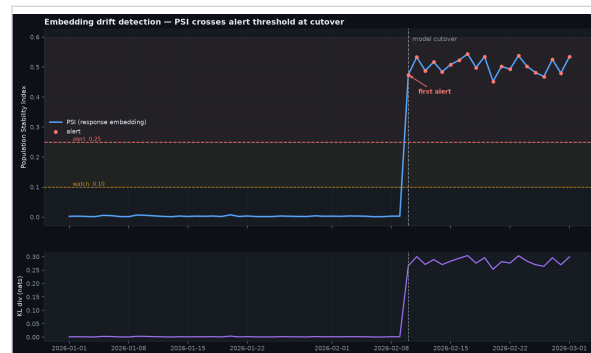
The evidence page shows that large trace volumes can still be queried quickly when storage is partitioned correctly. PSI highlights injected drift cleanly, which gives the quality layer an early warning signal. The visual scorecards make per-model trade-offs legible for both platform and risk stakeholders.

6.2 Test Result Screenshots

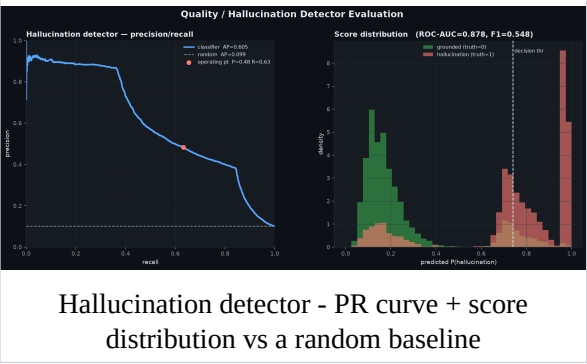
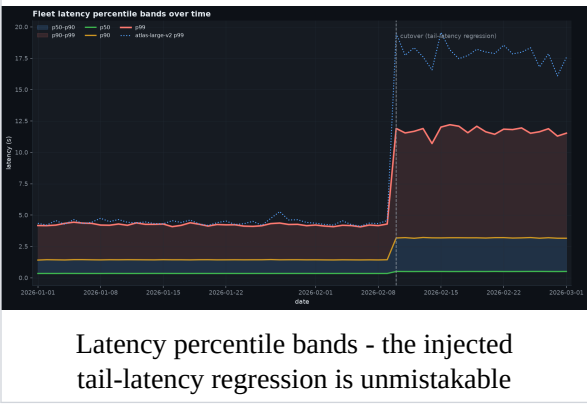
The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.



Fleet dashboard - KPI tiles, latency-over-time, spend/tokens by model



Drift detection - PSI crosses the alert threshold exactly at the model cutover



6.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	21 tests committed in repository validation flow.
Benchmark results	Limited	No benchmark markdown tables were present; discussion relies on screenshots and h
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

7. Discussion

The project demonstrates that a local, partitioned analytics path can still support fleet-level insight. The same trace lake supports operational reporting, shift detection, and model-quality scorecards in one review flow. The evidence page shows that large trace volumes can still be queried quickly when storage is partitioned correctly. PSI highlights injected drift cleanly, which gives the quality layer an early warning signal. The visual scorecards make per-model trade-offs legible for both platform and risk stakeholders. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

8. Conclusion

This project document concludes that LLM Observability & Eval is best understood as a complete technical implementation supported by repository-native evidence. This brief documents an observability stack for LLM fleets where trace analytics, cost monitoring, drift detection, and hallucination checks share the same data backbone instead of living in disconnected tools.

9. Future Work

- Add human-review annotations and slice-aware eval routing by tenant or product workflow.
- Introduce alert policies that combine drift, cost, and quality instead of triggering on single metrics.
- Extend the dashboard into a comparative release-review workflow for prompt or model cutovers.

10. References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/scaling.md - benchmark or result artefact used in the analysis section.
4. tests/ - automated validation suite used to support implementation claims.
5. assets/.gitkeep - generated screenshot evidence included in the report.
6. assets/dashboard.png - generated screenshot evidence included in the report.